

L01. What is sequential decision making?

Lecturer: Munther Dahleh

Scribe: Jinhee Won

1 What is reinforcement learning?

Reinforcement learning is the study of *sequential decision making*: an agent repeatedly takes actions, observes their consequences, and adapts its strategy over time.

Think about learning to ride a bicycle. When did you learn—at age two, three, or four? You almost certainly did not begin by writing down Newton’s laws or analyzing the stability of the bicycle. Nevertheless, you learned. At every moment, you adjusted your behavior in response to what happened, with a simple objective in mind: do not fall.

In reinforcement-learning language, avoiding a fall was part of your *reward*. Learning to ride a bicycle is therefore a sequential decision problem in which the learner changes its strategy after gaining experience.

Now contrast this with learning to fly an airplane. Why do we not learn only by trial and error, following the same principle? The cost of failure dominates. In safety-critical settings, unrestricted exploration is unacceptable. We may still learn from data or practice in a simulator, but safety constraints, models, and carefully designed procedures must limit what can be tried in the real system. Reinforcement learning cannot be applied “willy-nilly.” This is one reason safe exploration is a central problem in reinforcement learning.

Aside

This distinction motivates the field of *safe reinforcement learning*, which studies how an agent can improve its behavior while satisfying safety requirements during learning and deployment (García and Fernández, 2015; Brunke et al., 2022).

2 Rewards

So, what defines how we improve? In the bicycle case, things are a little more obvious. Maybe you move your handle to the right, or sit up a little straighter, and you get direct feedback on how much longer you can stay up without falling.

However, consider a game of chess. The goal of the game is to not lose. You know that blundering your queen is probably a bad idea. But to anyone who has only heard of the rules as don’t lose, that might not be as obvious. This means that you internally have some sort of idea of what the intermediary rewards are. One solution that people consider to address this problem is reward engineering. In the game of chess, that looks like mapping a pawn to one point and a queen to nine, and evaluating the position and action with a potential reward.

Aside

Another possible way to think about the chess problem is keeping the sparse rewards but assigning weights to past specific actions. Determining which earlier actions caused a delayed outcome is known as the *temporal credit-assignment problem*. When useful rewards are also rare, the problem is closely connected to exploration in sparse-reward environments. For surveys of temporal credit assignment and exploration methods, see (Pignatelli et al., 2024; Ladosz et al., 2022).

In another case, we can apply reinforcement learning on top of a control system, such as self-driving cars. We can learn the behaviors of the drivers around us to build a reward function that prioritizes safety and comfort. However, this reward design can get.. interesting. See this paper where that dilemma truly comes into play (Greene, 2016). Although humans morally believe that they should sacrifice themselves rather than hit and kill a pedestrian, in practice, if asked if they would buy a car that would follow that procedure, they remain hesitant and unlikely to do so. As humans, we don't have to make these decisions unless it happens to us. In machines however, these reward prioritization have to be programmed in.

Stepping back from ethical implications, reward design can also get complicated when we have to make sequential decisions that prioritize multiple objectives. For example, in hospitals, they have to improve health while also cutting costs.

Aside

Multi-objective RL spans a wide range of approaches, with work studying trade-offs, preference-dependent solutions, and sets of Pareto-optimal policies (Rojers et al., 2013; Hayes et al., 2022).

Recently, another example of reward functions in reinforcement learning comes with the idea of alignment. We "align" the answers of models between two pairs that humans prefer. In an LLM, let's say we get a "hallucination," or an answer we don't like or is maybe incorrect. We use RL to prioritize producing answers we prefer between two examples.

Aside

Reinforcement learning from human feedback (RLHF) is a very active field that seeks to address alignment and hallucinations, especially in LLMs (Christiano et al., 2017; Wirth et al., 2017).

Finally, let's say we design a good enough reward function. That might actually not be good enough. Because the only thing that reinforcement learning knows to optimize for is the reward function, we may run into issues of reward hacking, or systems suddenly finding loopholes that we may not have anticipated, often termed "reward hacking."

We can continue on and on with these examples, with areas in RL spanning a vast range of work beyond the examples here. What this really goes to show is how important problem formulation and setup is within RL applications in order to solve the problem you want to solve.

3 Setup and Notation

Now, we need a way to formalize how we discuss sequential decision making. We can state ourself as the agent, that follows a certain strategy (policy π). This policy advises us to take a certain action (a_t) given our observation (o_t) of the environment. Our strategy is also updated based on the reward we get (r_t) from taking that action. On the environment side, there is a state (s_t) that gets updated by the action according to a transition function (f, P) it follows.

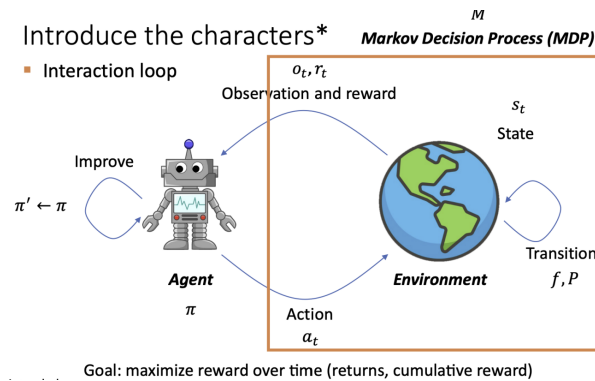


Figure 1: The relevant "characters" in sequential decision making.

Going back to the bicycle example, the state might describe the rider's position, speed, and lean angle. The rider observes some information about this state and takes an action, such as steering or shifting their weight. If that action keeps the bicycle stable for another five or ten seconds, the rider receives a favorable reward. Through repeated interaction, the rider learns to prioritize actions that work well in particular situations. In this way, the rider gradually improves their policy π .

3.1 This course

As the agent moves through the state space, the transition model determines the next state. The dynamics may be known or unknown.

If the model is known, we can pose an optimal-control problem and use tools such as Bellman equations and dynamic programming. We begin with this known-model case to build up the intuition and reasoning behind why RL works.

We then slowly start removing information. Reinforcement learning becomes central when the transition or reward model is unknown and must be learned through interaction. Later, we can also remove full access to the state itself: when the agent receives only partial observations, the model becomes a partially observable Markov decision process (POMDP).

3.2 State Space

Let us discuss the state space in more detail. At time t , the state of the environment is denoted by

$$s_t \in \mathcal{S},$$

where $\mathcal{S}$ is the set of all possible states. Ignoring actions for now, the state evolves according to some dynamics,

$$s_{t+1} = f_t(s_t, w_t),$$

where a_t is the action taken by the agent and w_t represents uncertainty in the environment. We describe this uncertainty by specifying a probability distribution for w_t .

Does this system have *Markov* transition dynamics?

Maybe. It seems to only depend on the state prior, s_t . However, we cannot be sure as we do not know what the uncertainty in the environment depends on. For all we know, it could depend on a prior state.

3.3 IID Noise, Markov Property

As a simple example, suppose the state evolves according to

$$s_{t+1} = \frac{1}{2}s_t + w_t,$$

where the disturbances w_t are independent and identically distributed. In this case, the distribution of the next state depends only on the current state s_t . We do not need to know $s_{t-1}, s_{t-2}, \dots$ separately.

Notice that we cannot fully predict the value of s_{t+1} before observing w_t . What we can predict is its probability distribution. For example, if w_t has mean zero, then

$$s_t = \left(\frac{1}{2}\right)^t s_0 + \sum_{\tau=0}^{t-1} \left(\frac{1}{2}\right)^{t-\tau-1} w_\tau.$$

The second term is a weighted sum, or convolution, of the previous disturbances. Thus, the current state depends on the history of the system.

Nevertheless, we do not need to know each previous disturbance separately to predict the distribution of the next state. Once s_t is known,

$$s_{t+1} = \frac{1}{2}s_t + w_t.$$

Therefore, if w_t is independent of the past, then

$$\Pr(s_{t+1} \mid s_0, \dots, s_t) = \Pr(s_{t+1} \mid s_t).$$

This is the Markov property: the current state summarizes all information from the past that is needed to predict the future.

References

- Brunke, L., Greeff, M., Hall, A. W., Yuan, Z., Zhou, S., Panerati, J., and Schoellig, A. P. (2022). Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(1):411–444.
- Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. (2017). Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30.
- García, J. and Fernández, F. (2015). A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480.

- Greene, J. D. (2016). Our driverless dilemma. *Science*, 352(6293):1514–1515.
- Hayes, C. F., Rădulescu, R., Bargiacchi, E., Källström, J., Macfarlane, M., Reymond, M., Verstraeten, T., Zintgraf, L. M., Dazeley, R., Heintz, F., Howley, E., Irissappane, A. A., Mannion, P., Nowé, A., Ramos, G., Restelli, M., Vamplew, P., and Roijers, D. M. (2022). A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1):1–59.
- Ladosz, P., Weng, L., Kim, M., and Oh, H. (2022). Exploration in deep reinforcement learning: A survey. *Information Fusion*, 85:1–22.
- Pignatelli, E., Ferret, J., Geist, M., Mesnard, T., van Hasselt, H., Pietquin, O., and Toni, L. (2024). A survey of temporal credit assignment in deep reinforcement learning. *Transactions on Machine Learning Research*.
- Roijers, D. M., Vamplew, P., Whiteson, S., and Dazeley, R. (2013). A survey of multi-objective sequential decision-making. *Journal of Artificial Intelligence Research*, 48:67–113.
- Wirth, C., Akrou, R., Neumann, G., and Fürnkranz, J. (2017). A survey of preference-based reinforcement learning methods. *Journal of Machine Learning Research*, 18(136):1–46.